Summarize

# Afterword: Bringing It All Together

This book has covered a lot of ground, and I've shared a lot of advice along the way. Given the breadth of coverage, I thought it sensible to summarize some of my key advice regarding microservice architectures. For those of you who have read the whole book, this should be a great refresher. For those of you who are impatient and jumped to the end, be aware that there is a *lot* of detail behind this advice, and I'd urge you read up on the detail behind some of these ideas rather than just adopting these ideas blindly.

With all that said, I'm aiming to keep this last chapter as brief as possible, so let's get started.

## What Are Microservices?

As introduced in Chapter 1, *microservices* are a type of service-oriented architecture that focuses on independent deployability. *Independent deployability* means that you can make a change to a microservice, deploy that microservice, and release its functionality to the end users without requiring other microservices to change. Getting the most out of a microservice architecture means embracing this concept. Normally, each microservice is deployed as a process, with communication with other microservices being done over some form of network protocol. It's common to deploy multiple instances of a microservice, perhaps so that you can provide more scale, or else to improve robustness by having redundancy.

To deliver independent deployability, we need to make sure when changing one microservice that we don't break interactions with other microservices. This requires that our interfaces with other microservices are stable, and that changes be made in a backward-compatible way. *Information hiding*, which I expanded on in Chapter 2, describes an approach in which as much information as possible (code, data) is hidden behind an interface. You should expose only the bare minimum over your service interfaces to satisfy your consumers. The less you expose, the easier it is to ensure the changes you make are

going to be backward compatible. Information hiding also allows us to make technological changes within a microservice boundary in a way that won't impact consumers.

One of the key ways we implement independent deployability is by *hiding the database*. If a microservice needs to store state in a database, this should be entirely hidden from the outside world. Internal databases should not be directly exposed to external consumers, as this causes too much coupling between the two, which undermines independent deployability. In general, avoid situations in which multiple microservices all access the same database.

Microservices work very well with *domain-driven design* (DDD). DDD gives us concepts that help us find our microservice boundaries, with the resulting architecture being aligned around the business domain. This is extremely helpful in situations where organizations are creating more business-centric IT teams. With a team focused on one part of the business domain, it can now take ownership of the microservices that match this part of the business.

## Moving to Microservices

Microservices bring a *lot* of complexity—enough complexity that the reasons for using them need to be seriously considered. I remain convinced that a simple single-process monolith is a totally sensible starting point for a new system. Over time, though, we learn things, and we start to see the ways in which our current system architecture is no longer fit for purpose. At that point, looking to change is appropriate.

It is important to understand *what* you are trying to get out of a microservice architecture. What is the goal? What positive outcome do you expect a shift to microservices to bring? The outcome you are aiming for will directly impact *how* you break your monolith apart. If you are trying to change your system architecture to better handle scale, you'll end up making different changes than if your main driver is to improve organizational autonomy. I cover this more in Chapter 3, and in even more detail in my book *Monolith to Microservices*.

Many of the problems with microservices are evident only after you hit production. Therefore, I strongly recommend an incremental, evolutionary decomposition of an existing monolith rather than a "big bang" rewrite. Identify a microservice you want to create, extract the appropriate functionality from the monolith, deploy the new microservice into production, and start using it in anger. Based on that, you'll see if you are helping move toward your goal, but you'll also learn a lot that will make the next microser-

vice extraction easier—or perhaps it will suggest to you that microservices might not be the way forward after all!

# Communication Styles

We summarized the main forms of inter-microservice communication in Chapter 4, shared again in Figure E-1. This isn't meant to be a universal model but is intended to just give an overview of the different types of communication that are most common.

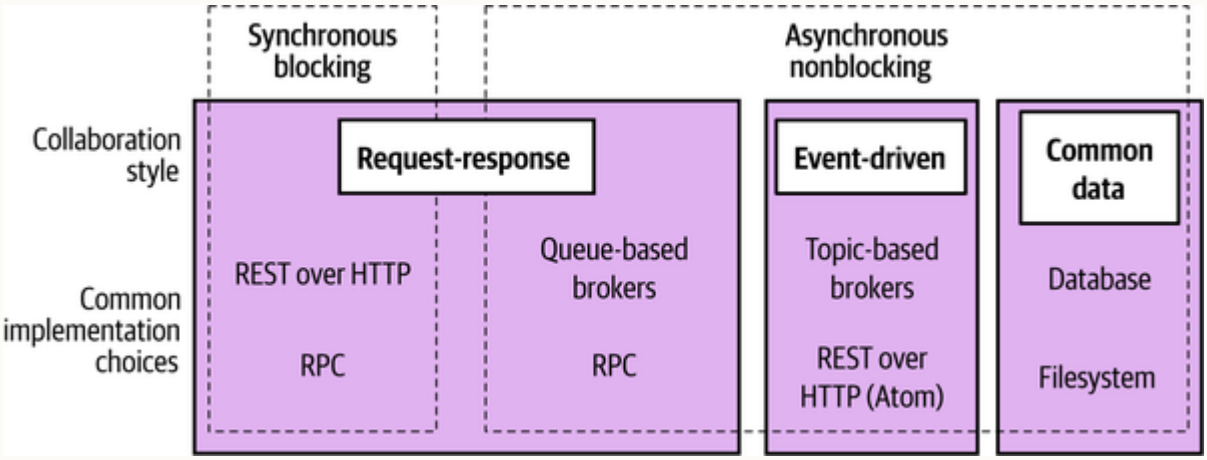



Figure E-1. Different styles of inter-microservice communication along with example implementing technologies

With *request-response* communication, a microservice sends a request to a downstream microservice and expects a response. With synchronous request-response, we would expect the response to come back to the microservice instance that sent the request. With asynchronous request-response, it's possible for the response to come back to a different instance of the upstream microservices.

With *event-driven* communication, a microservice emits an event, and other microservices, if they are interested in that event, can react to it. Events are just statements of fact—information that is shared about something that has happened. With event-driven communication, a microservice doesn't tell another microservice what to do; it just shares events. It's up to downstream microservices to make a judgment call as to what they do with that information. Event-driven communication is by definition asynchronous in nature.

One microservice may communicate over more than one protocol. For example, in Figure E-2 we see a `Shipping` microservice providing a REST interface for request-response interaction, which also fires events when changes are made.

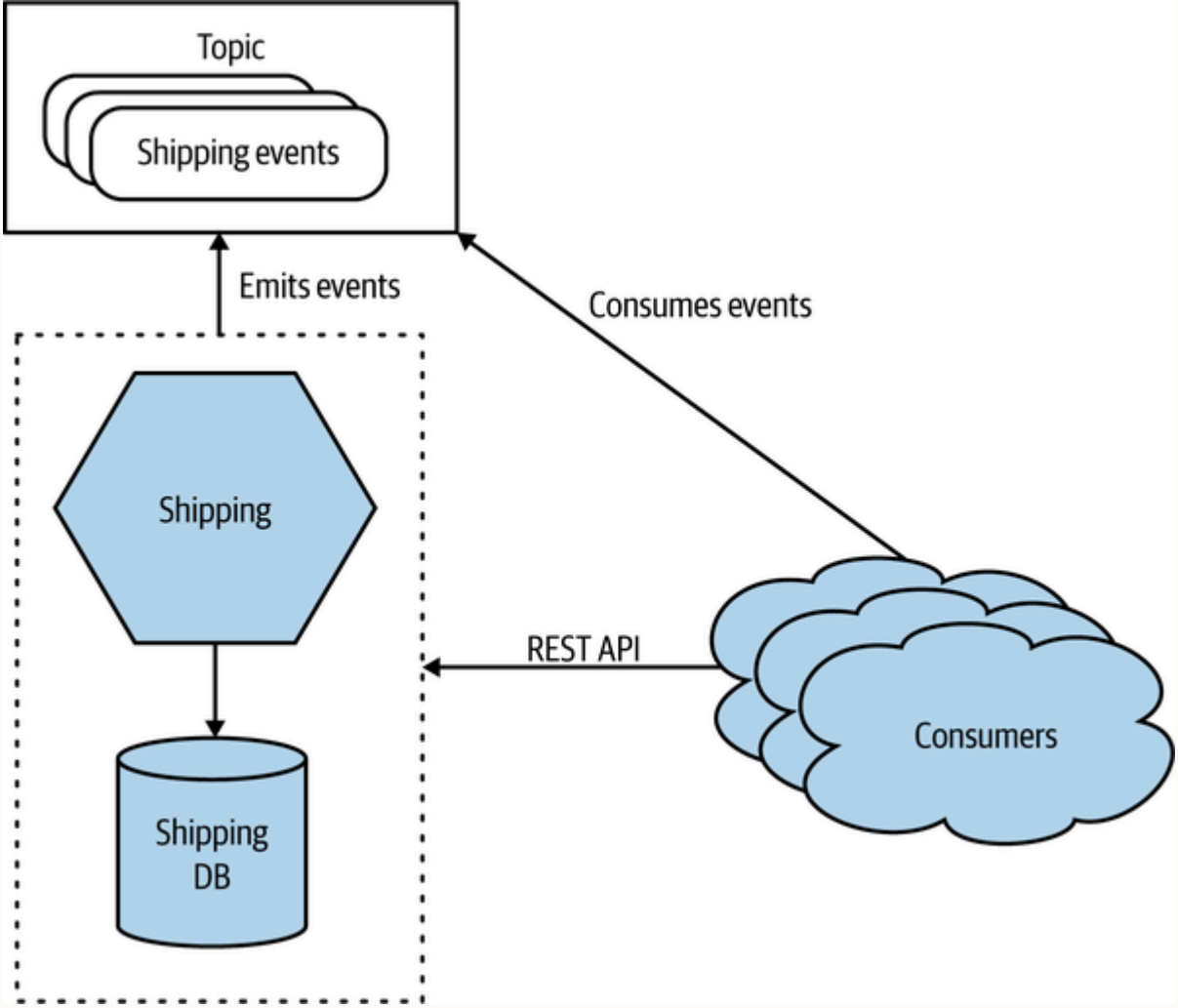



Figure E-2. A microservice exposing its functionality over a REST API and a topic

Event-driven collaboration can make it easier to build more loosely coupled architectures, but it can require more work to understand how the system is behaving. This type of communication also often requires the use of specialist technology such as message brokers, which can further complicate matters. If you can use a fully managed message broker, that can help lower the cost of these types of systems.

Request-response and event-driven interaction models both have their place, and often which one you use will be a personal preference. Some problems just fit one model more than another, and it's common for a microservice architecture to have a mix of styles.

## Workflow

When looking to get multiple microservices collaborating to perform some overarching operation, look to explicitly model the process using *sagas*, a topic we explored in Chapter 6.

In general, distributed transactions should be avoided in situations where you can use a saga instead. Distributed transactions add significant complexity to systems, have prob-

lematic failure modes, and often don't deliver what you expect even when they work. Sagas are in virtually all cases a better fit for implementing business processes that span multiple microservices.

There are two different styles of sagas to consider: *orchestrated* sagas and *choreographed* sagas. Orchestrated sagas use a centralized orchestrator to coordinate with other microservices and ensure that things are done. In general, this is a simple and straightforward approach, but the central orchestrator can end up doing too much if you aren't careful, and it can become a source of contention when multiple teams are working on the same business process. With choreographed sagas, there is no central coordinator; instead, the responsibility for the business process is distributed into a number of collaborating microservices. This can be a more complex architecture to implement, and it requires more work to ensure that the right things are happening, but on the flip side it is much less prone to coupling and works well for multiple teams.

Personally, I love choreographed sagas, but then I've used them a lot and made a lot of mistakes implementing them. My general advice is that orchestrated sagas work fine when a single team owns responsibility for the whole process, but they become more problematic with multiple teams. Choreographed sagas can justify their increased complexity in situations in which multiple teams are expected to collaborate on a process.

## Build

Each microservice should have its own build, its own CI pipeline. When I make a change to a microservice, I expect to be able to build that microservice by itself. Avoid situations in which you have to build all your microservices together, as this makes independent deployability much harder.

For reasons outlined in Chapter 7, I am not a fan of monorepos. If you really want to use them, then please understand the challenges they cause around clear lines of ownership and potential complexity of builds. But definitely make sure that, whether you use a monorepo or a multirepo approach, each microservice has its own CI build process that can be triggered independently of any other builds.

## Deployment

Microservices are normally deployed as a process. This process can be deployed onto a physical machine, a virtual machine, a container, or an FaaS platform. Ideally, we want

microservices to be as isolated from one another as possible in a deployed environment. We don't want a situation in which one microservice using up lots of computing resources can impact a different microservice. In general, this means we want to have each microservice using its own ring-fenced operating system and set of computing resources. Containers are especially effective at giving each microservice instance its own ring-fenced set of resources, making them a great choice for microservice deployments.

Kubernetes can be very useful if you are looking to run container workloads across multiple machines. It's not something I'd recommend for just a few microservices, as it brings with it its own sources of complexity. Where possible, use a managed Kubernetes cluster, as this allows you to avoid some of this complexity.

FaaS is an interesting emerging pattern in deploying code. Rather than having to specify how many copies of something you want, you just give your code to the FaaS platform and say, "When this happens, run this code." This is *really* nice from a developer point of view, and I think an abstraction like this is likely the future for a large amount of server-side development. The current implementations aren't without problems, though. In terms of microservices, deploying a whole microservice as a single "function" on a FaaS platform is a totally fine way to start.

One final note: separate in your mind the concepts of deployment and release. Just because you've deployed something to production doesn't mean it has to be released to your users. By separating these concepts, you open up the opportunity to roll out your software in different ways—for example, by using canary releases or parallel runs. All of this and more is covered in depth in Chapter 8.

## Testing

It makes a lot of sense to have a suite of automated functional tests to give you fast feedback on the quality of your software before the users see it, and this is absolutely something you should do. Microservices give you a lot of options in terms of the different types of tests you can write, as we explored in Chapter 9.

When compared to other types of architectures, though, end-to-end tests can be especially problematic for microservice architectures. They can end up being more expensive to write and maintain for microservice architectures than for simpler nondistributed architectures, and the tests themselves can end up having a lot more failures that don't necessarily point to a problem with your code. End-to-end tests that span multiple teams are particularly challenging.

Over time, look to reduce your reliance on end-to-end tests; consider replacing some of the effort put into this form of testing with consumer-driven contracts, schema compatibility checking, and testing in production. These activities can deliver much more effectively than end-to-end tests on quickly catching issues on more distributed systems.

## Monitoring and Observability

In Chapter 10, I explained how *monitoring* is an activity, something we do *to* a system, but that focusing on an activity rather than an outcome is problematic, a thread that has run through this book. Instead, we should focus on the *observability* of our systems. Observability is the extent to which we can understand what a system is doing by examining external outputs. Making a system that has good observability requires that we build this thinking into our software and ensure that the right types of external outputs are available.

Distributed systems can fail in strange ways, and microservices are no different. We cannot predict all causes of system failure, so it can be hard to know what information we'll need ahead of time to diagnose and fix issues. Using tooling that can help you interrogate these external outputs in ways that you cannot expect becomes increasingly important. I suggest that you look at tools like Lightstep and Honeycomb that were built with this thinking in mind.

Finally, as your system grows in scale, it becomes more and more likely that there will always be an error somewhere. But in a large-scale system, one machine having a problem isn't necessarily cause for everyone to jump into action, nor should this necessarily result in a rude awakening for anyone at 3 a.m. Using "test in production" techniques like parallel runs and synthetic transactions can be much more effective for picking up problems that might actually be impacting the end users.

## Security

Microservices give us more opportunity to defend our application in depth, which in turn can lead to more secure systems. On the other hand, they often have a larger attack surface area, which can leave us more exposed to attack! This balancing act is why it's so important to have a holistic understanding of security, something I shared in Chapter 11.

With more information flowing over networks, it becomes more important to consider the protection of data in transit. The increased number of moving parts also means that

automation is a vital part of microservice security. Managing patching, certificates, and secrets using manual, error-prone processes can leave you vulnerable to attack. So use tools that allow for ease of automation.

JWTs can be used to decentralize authorization logic in a way that also avoids the need for additional round trips. This can help protect you from issues like the confused deputy problem, while at the same time ensuring your microservice can run in a more independent fashion.

Finally, increasing numbers of people are adopting a *zero-trust* mindset. With zero trust, you operate as though your system has already been compromised and you need to build your microservices accordingly. It may seem like a paranoid stance, but I'm increasingly of the opinion that embracing this principle can actually simplify how you view the security of your system.

# Resiliency

In Chapter 12, we looked at resiliency as a whole, and I shared with you the four key concepts that need to be considered when thinking about resiliency:

*Robustness*

: The ability to absorb expected perturbation

*Rebound*

: The ability to recover after a traumatic event

*Graceful extensibility*

: How well we deal with a situation that is unexpected

*Sustained adaptability*

: The ability to continually adapt to changing environments, stakeholders, and demands

Taken as a whole, microservice architectures can help with some of these things (namely robustness and rebound), but as we see from this list, that by itself doesn't make you resilient. Much of being resilient is about team and organizational behavior and culture.

Fundamentally, you have to explicitly do things to make your application more robust. Robustness isn't free—microservices give us the option to improve the resiliency of our systems, but we have to make that choice. For example, we have to understand that any call we make to another microservice might fail, that machines might die, and that bad things happen to good network packets. Stability patterns like bulkheads, circuit breakers, and properly configured time-outs can help greatly.

## Scaling

Microservices give us a number of different ways to scale an application. In Chapter 13, I explore the four axes of scaling, which I share below:

*Vertical scaling*

: In a nutshell, this means getting a bigger machine.

*Horizontal duplication*

: Having multiple things capable of doing the same work.

*Data partitioning*

: Dividing work based on some attribute of the data, e.g., customer group.

*Functional decomposition*

: Separation of work based on the type, e.g., microservice decomposition.

With scaling, do the easy stuff first. Vertical scaling and horizontal duplication are quick and easy compared with the other two axes presented here. If they work, great! If not, you can look at the other mechanisms. It's common as well to mix the different types of scaling—partitioning your traffic based on customers, for example, and then having each partition scaled horizontally.

## User Interfaces

All too often, the user interface is an afterthought when it comes to system decomposition—we break apart our microservices but leave a monolithic user interface. This in turn leads to the problems of having separate frontend and backend teams. Instead, we want stream-aligned teams, where one team owns all the functionality associated with an

end-to-end slice of user functionality. To make that change happen and get rid of siloed frontend and backend teams, we need to break apart our user interfaces.

In Chapter 14, I share how we can use *micro frontends* to deliver decomposed user interfaces using single-page app frameworks like React. User interfaces often face problems in terms of the number of calls they need to make, or because they need to perform call aggregation and filtering to suit mobile devices. The *backend for frontend* (BFF) pattern can help provide server-side aggregation and filtering in these situations, although if you are able to use GraphQL, you may be able to sidestep the use of BFFs.

# Organization

In Chapter 15, we looked at the shift away from horizontally aligned, siloed teams toward team structures that are organized around end-to-end slices of functionality. These *stream-aligned* teams, as the authors of *Team Topologies* describe them, are supported by *enabling* teams, as Figure E-3 shows. Enabling teams will often have a specific cross-cutting focus, such as focusing on security or usability, and support the stream-aligned teams in these aspects.

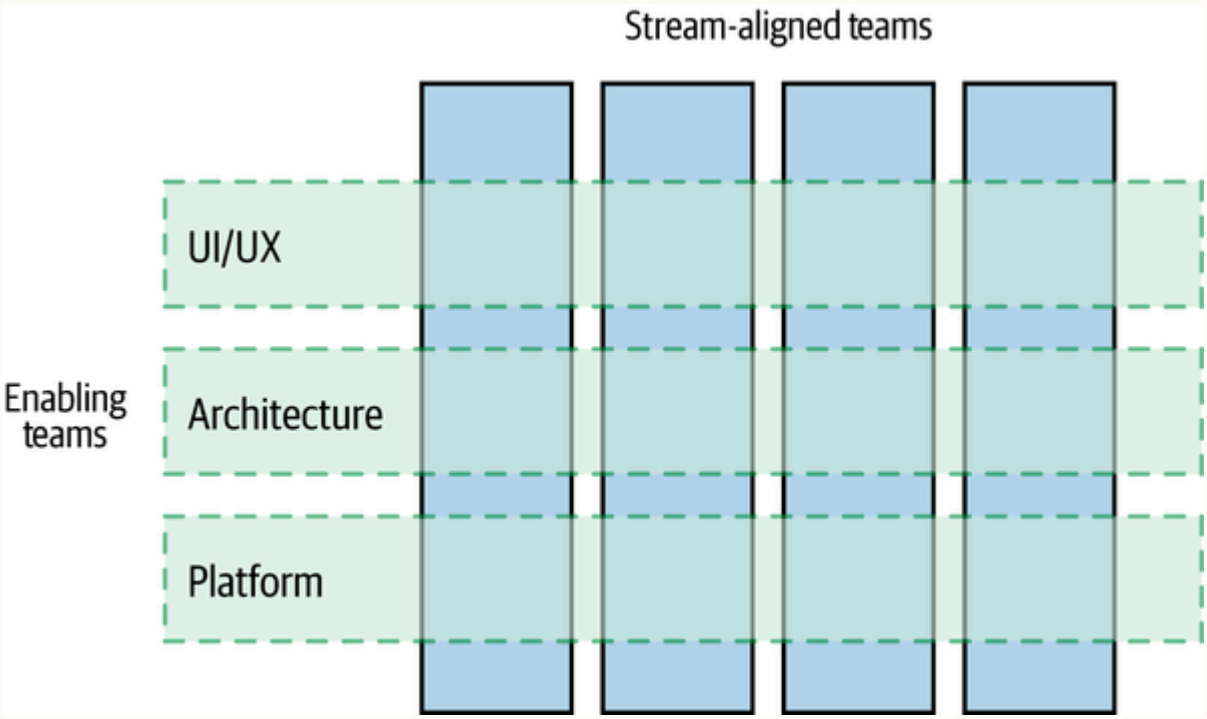



Figure E-3. Enabling teams support multiple stream-aligned teams

Making these stream-aligned teams as autonomous as possible means that they need self-service tools to avoid having to constantly ask other teams to do things for them. As part of this, a platform can be incredibly useful. It's important, though, that we see a platform as a type of *paved road*—that is, as something that makes it easy to do the right thing, without requiring that it *must* be used. Making a platform optional ensures that

making the platform easy to use remains a key focus of the team owning it, while also allowing teams to make a different choice when warranted.

## Architecture

It's important that we don't see the architecture of our system as fixed and unchanging. Instead, we should view our system architecture as something that should be able to continually change as circumstances require. For you to get the most out of microservice architectures, moving to an organization where more autonomy is pushed into teams means that responsibility for the technical vision needs to become a more collaborative process. The architect sitting in an ivory tower will either be a significant blocker to a microservice architecture or else will become an ignored irrelevance.

The role of shepherding the architecture of a system can be entirely distributed into the teams, and at a certain level of scale this can work well. However, as the organization grows, having people with dedicated time to look across the system as a whole becomes essential. Call them principal engineers, technical product owners, or architects, it doesn't really matter—the role that they need to play is the same. As I showed in Chapter 16, architects in a microservice organization need to support teams, connect people, spot patterns emerging, and spend enough time embedded with teams to see how the big picture stuff plays out in reality.

## Further Reading

Throughout this book, I have referenced many papers, presentations, and books that I have learned a great deal from, and I have made sure to list them in Bibliography. Since the first edition, though, the two books that have had the biggest impact on my thinking, and as a result have been referenced extensively in this new edition, are worth calling out here as "must reads." The first is *Accelerate* by Nicole Forsgren, Jez Humble, and Gene Kim. The second is *Team Topologies* by Matthew Skelton and Manuel Pais. These two books are, in my opinion, the two most useful books on software development written in the last ten years, and I consider them essential reading whether you are into microservices or not.

As a companion to this book, my own *Monolith to Microservices* goes into more depth on how to break apart existing system architectures.

## Looking Forward

In the future, I suspect the technology that makes microservices easier to build and run will continue to improve, and I am especially keen to see what the second- (and third-) generation FaaS products look like. Whether or not FaaS takes off, Kubernetes will become even more widespread, even if it will increasingly be hidden behind more developer-friendly abstraction layers. Kubernetes has won, but in a way that I think most application developers shouldn't have to worry about. I remain very interested in seeing how Wasm changes how we think about deployments, and I still have a suspicion that unikernels may have a second coming as well.

Since this book's first edition, microservices have well and truly gone mainstream in a way that has surprised me, and that has concerned me too. It seems that a lot of people adopting microservices are doing so more because everyone else is doing it, rather than microservices being right for them. As such, I fully expect us to hear more horror stories about failed microservice implementations, which I will digest with relish to see what can be learned. I also fully expect a wider industry backlash against microservices at some point when the microservice disaster case studies reach critical mass. Applying critical thinking to work out what approach makes the most sense in any given situation isn't very sexy or marketable, and I don't expect that will change in a world where selling technology is more profitable than selling ideas.

I don't mean to sound pessimistic! We are, as an industry, still very young, and we are still finding our place in the world. The amount of energy and ingenuity that gets put into software development continues to keep me interested, and I can't wait to see what the next decade brings.

# Final Words

Microservice architectures give you more options, and more decisions to make. Making decisions in this world is a far more common activity than in simpler, monolithic systems. You won't get all of these decisions right, I can guarantee that. So, knowing you are going to get some things wrong, what are your options? Well, I would suggest finding ways to make each decision small in scope; that way, if you get one wrong, you impact only a small part of your system. Learn to embrace the concept of evolutionary architecture, in which your system bends and flexes and changes over time as you learn new things. Think not of big-bang rewrites, but instead of a series of changes made to your system over time to keep it supple.

I hope by now I've shared with you enough information and experiences to help you decide whether microservices are for you. If they are, I hope you think of this as a journey, not a destination. Go incrementally. Break your system apart piece by piece, learning as you go. And get used to it: in many ways, the discipline to continually change and evolve our systems is a far more important lesson to learn than any other I have shared with you through this book. Change is inevitable. Embrace it.

Previous chapter
< 16. The Evolutionary Architect

Next chapter
Bibliography >